Software Requirements Specification (SRS) & Engineering Design Document

Project Title: Vitalis: Predictive Analytics for Blood, Organ & Medical Supply Redistribution

1. Introduction & Academic Scope

1.1 Purpose

This document provides the formal Software Requirements Specification (SRS) for **Vitalis**, an event-driven, agentic clinical logistics platform designed to eliminate preventable mortality caused by localized blood and biological supply stockouts. It establishes the complete software architecture, interface definitions, database models, machine learning specifications, and verification protocols required for end-to-end development.

1.2 Academic Course Alignment (BCSE335L)

The technical deliverables of this project map directly to the BCSE335L syllabus modules:

- **Module 2 (Healthcare Foundations):** Normalizing raw medical telemetry and unstructured dispatch requests into standardized clinical codesets and Emergency Severity Index (ESI) categories.
- **Module 3 (Machine Learning Foundations):** Executing an end-to-end ML pipeline featuring data loading, cleaning, feature selection, and tree-like reasoning via XGBoost classifiers.
- **Module 4 (Measuring Healthcare Quality):** Integrating Medicare value-based purchasing programs—specifically Hospital Value-Based Purchasing (HVBP), Hospital Readmissions Reduction Program (HRR), and Hospital-Acquired Condition Reduction Program (HAC)—as algorithmic penalties.
- **Module 5 (Predictive Models in Healthcare):** Training rolling time-series models on the National Hospital Ambulatory Medical Care Survey (NHAMCS) emergency dataset to model clinical trauma spikes.
- **Module 6 (Healthcare Analytics Applications):** Tri-layer analytics implementation: Descriptive (Next.js Network Dashboard), Predictive (Prophet/XGBoost stockout forecasting), and Prescriptive (LangChain ReAct dispatch solver).
- **Module 7 (Emerging Technologies & Ethics):** IoT telematics for cold-chain transit tracking via React Native, backed by Human-in-the-Loop (HITL) ethical guardrails.

2. System Architecture & Component Interaction

Architecture Layer	Core Components & Tools	Responsibilities & Outputs
1. Unstructured Inputs	Radio Transcripts, SMS, Pager Blasts, EHR Alerts	Ingests raw natural language requests and dispatch triggers from medical channels.
2. Ingestion & Triage Service	JEV / Fine-Tuned NLP, Pydantic Schema Validator	Extracts clinical entities, enforces type validation, and emits Structured Request JSON.
3. Orchestration Gateway	FastAPI Gateway, LangChain ReAct Agent Core	Coordinates agent execution loop, routes API tasks, and manages end-to-end request state.
4. Agent Tooling & Data Services	• **Text-to-SQL Tool:** PostgreSQL / PostGIS Proximity Search • **Predictive Tool:** XGBoost / Prophet Model Inference • **Spatial Router Tool:** OSRM Distance Matrix & Viability Constraints	Queries donor inventory, computes donor stockout risk probabilities, and verifies transit window shelf-life.
5. Priority & Decision Verification	Prescriptive Priority Engine (Priority Index calculation with HAC quality penalties)	Ranks candidate transfer options and verifies decisions before queueing for approval.
6. Presentation & Execution Clients	• **Next.js Command Center:** GIS Topology Map, Explainable Reasoning Trace, HITL Approval • **React Native Courier App:** Route Telemetry, IoT Cold-chain, Chain-of-custody Barcode Scan	Provides operational oversight for human approval and mobile tools for courier navigation and cold-chain logging.

3. Detailed Functional Requirements (FR)

FR-1: Ingestion & Triage Service

- **Description:** The system must parse unstructured natural language requests containing medical jargon, regional colloquialisms, or radio abbreviations and output a strictly typed, validated emergency request payload.
- **Inputs:** Raw text string (body: str), request source identifier (source: Enum['radio', 'sms', 'ehr', 'manual']).
- **Processing Rules:**

1. The text is passed to the Jev System One classifier / fine-tuned NER extractor.
2. Extracted entities must be coerced into a Pydantic TriageExtractionPayload:
   - resource_type: Enum ['Whole Blood', 'PRBC', 'Platelets', 'FFP', 'Cryoprecipitate', 'Kidney', 'Liver', 'Heart'].
   - blood_group: Enum ['A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+', 'O-'].
   - units_requested: Positive integer ($1 \leq units \leq 50$).
   - urgency_tier: Integer corresponding to ESI triage scale ($1 = Immediate/Resuscitative, 5 = Non - Urgent$).

- **Outputs:** Validated JSON payload passed to the FastAPI Agent Gateway.

FR-2: Predictive Stockout & Consumption Engine

- **Description:** The system must forecast daily demand and compute the stockout probability for every registered facility to prevent predatory drawdowns during emergencies.
- **Processing Rules:**

1. **Demand Forecasting:** Meta Prophet models execute nightly per facility using historical time-series consumption curves derived from the NHAMCS dataset. Output includes point estimates and 80% uncertainty intervals ($\hat{y}_t, \hat{y}_{lower}, \hat{y}_{upper}$) for horizons $t \in [1, 14]$ days.
   Depletion Guardrail (XGBoost): When facility F_i is evaluated as a candidate to donate k units, an XGBoost classifier computes the 72-hour depletion risk:
   $$\text{Risk}_{\text{stockout}}(F_i, k) = P(Y = 1 \mid \vec{x})$$
   Where feature vector $\vec{x}$ includes:

$$\vec{x} = \left[current_inventory - k, \hat{y}_{t\leq 3}, HAC_score, trauma_level, burn_rate_{7d}\right]$$

2.

- **Guardrail Condition:** If $Risk_{stockout}(F_i, k) > 0.40$, the agent disqualifies facility F_i from being an outbound donor, preventing secondary stockouts.

FR-3: Agentic Dispatch Coordinator (LangChain ReAct)

- **Description:** A multi-tool LangChain ReAct agent must dynamically synthesize inventory availability, safety guardrails, transit viability, and Medicare quality penalties into a ranked list of transfer candidates.

- **Tool Execution Loop:**

1. Tool_PostGIS_Inventory: Performs a spatial range search (ST_DWithin) combined with SQL inventory filtering to return facilities within a viable radius holding unexpired stock.
2. Tool_Stockout_Guardrail: Calls the trained XGBoost model for each candidate to calculate the source depletion risk.
3. Tool_Viability_Calculator: Evaluates transportation time against biological shelf-life:
   - Platelets: Maximum 5 days shelf life; continuous agitation constraints.
   - PRBC (Packed Red Blood Cells): 35 to 42 days at 1°C–6°C.
   - Hearts/Lungs: Viability window $\leq 4hours$.
   - Kidneys: Viability window $\leq 24hours$.

   Prescriptive Ranking Equation: Candidates meeting hard viability filters are scored using the **Priority Index (PI)**:

$$PI = \frac{(UrgencyScore)\times(SurplusRatio)}{Distance(km)\times(1+RiskofSourceStockout)\times(QualityPenalty)}$$

-
  - $UrgencyScore = 6 - urgency_tier$ (ESI 1 scores 5; ESI 5 scores 1).
  - $SurplusRatio = \frac{AvailableUnits}{RequestedUnits}$.
  - $QualityPenalty = 1.0 + \max(0, \frac{HAC_Threshold - Quality_Score}{100})$.
- **Outputs:** Synthesized dispatch recommendation accompanied by an explainable natural-language reasoning trace.

FR-4: Command Center & HITL Interface

- **Description:** A Next.js web application providing operational command staff with regional inventory heatmaps, active stockout alerts, and an approval queue.
- **HITL Enforcement:** The system **must not** emit dispatch alerts to couriers autonomously. Every transfer recommendation remains in a pending_approval state until an authorized medical logistics coordinator confirms the transfer via an authenticated action.

FR-5: Courier Logistics & Cold-Chain Telemetry

- **Description:** A cross-platform mobile client (React Native) for logistics drivers that handles real-time transit telemetry, biological chain-of-custody tracking, and sensor logging.
- **Processing Rules:**

1. The app polls device location every 15 seconds during active delivery missions.
2. The driver must log and confirm cold-box temperature sensor values at dispatch pickup and delivery handoff.
3. If temperature exceeds biological tolerances ($> 6^\circ\text{C}$ for RBC, $> 24^\circ\text{C}$ for platelets), an automatic critical excursion alert is broadcast to the Next.js Command Center.

4. Database Schema (PostgreSQL / PostGIS / Supabase DDL)

The database must be initialized with the following structure, enforcing referential integrity, spatial geometries, and indexed query patterns:

SQL

```
-- Enable PostGIS and UUID Extensions
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "postgis";

-- 1. Facilities: Registered hospitals, blood banks, and municipal
transit depots
CREATE TABLE facilities (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name VARCHAR(255) NOT NULL,
    facility_type VARCHAR(50) NOT NULL CHECK (facility_type IN
('hospital', 'blood_bank', 'transit_hub')),
    trauma_level INT CHECK (trauma_level BETWEEN 1 AND 4),
    quality_score NUMERIC(5, 2) NOT NULL DEFAULT 100.00, --
Medicare HVBP/HAC Composite (0-100)
    location GEOMETRY(Point, 4326) NOT NULL,             -- WGS84
Spatial Coordinate
    contact_phone VARCHAR(50) NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 2. Inventory: Itemized biological product batches with
expiration tracking
CREATE TABLE inventory (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    facility_id UUID NOT NULL REFERENCES facilities(id) ON DELETE
CASCADE,
    resource_type VARCHAR(50) NOT NULL CHECK (
        resource_type IN ('Whole Blood', 'PRBC', 'Platelets',
'FFP', 'Cryoprecipitate', 'Kidney', 'Liver', 'Heart')
    ),
    blood_group VARCHAR(5) NOT NULL CHECK (
        blood_group IN ('A+', 'A-', 'B+', 'B-', 'AB+', 'AB-', 'O+',
'O-', 'N/A')
    ),
    units_available INT NOT NULL CHECK (units_available >= 0),
    storage_temp_celsius NUMERIC(4, 2) NOT NULL,
    expiry_timestamp TIMESTAMPTZ NOT NULL,
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
```

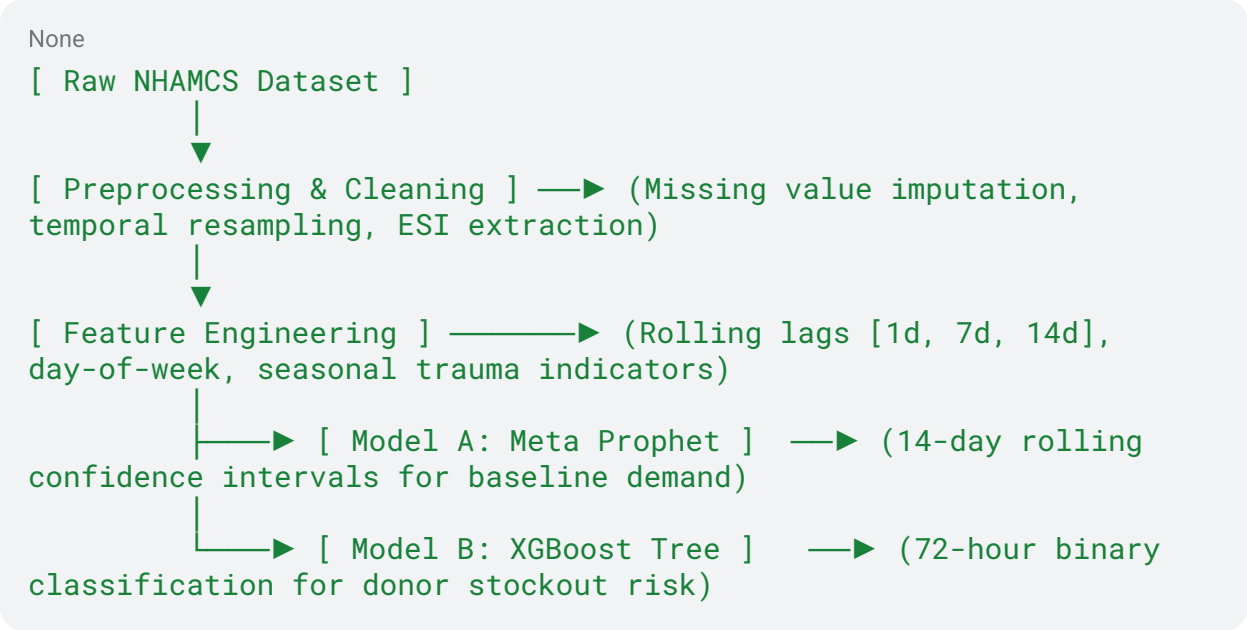
```
);

-- 3. Transfer Requests: Dispatch transactions, priority scoring,
and HITL lifecycle
CREATE TABLE transfer_requests (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    origin_facility_id UUID REFERENCES facilities(id),
    destination_facility_id UUID NOT NULL REFERENCES
facilities(id),
    resource_type VARCHAR(50) NOT NULL,
    blood_group VARCHAR(5) NOT NULL,
    units_requested INT NOT NULL CHECK (units_requested > 0),
    urgency_tier INT NOT NULL CHECK (urgency_tier BETWEEN 1 AND 5),
    agent_reasoning_trace TEXT,
    priority_score NUMERIC(8, 4),
    status VARCHAR(50) NOT NULL DEFAULT 'pending_approval' CHECK (
        status IN ('pending_approval', 'approved', 'in_transit',
'delivered', 'rejected', 'aborted')
    ),
    approved_by UUID, -- References authenticated admin user
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- 4. Cold Chain Telemetry: IoT monitoring during transit
CREATE TABLE transit_telemetry (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    transfer_id UUID NOT NULL REFERENCES transfer_requests(id) ON
DELETE CASCADE,
    recorded_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    current_location GEOMETRY(Point, 4326),
    temperature_celsius NUMERIC(4, 2) NOT NULL,
    excursion_flag BOOLEAN NOT NULL DEFAULT FALSE
);

-- Spatial and Performance Indexes
CREATE INDEX idx_facilities_location ON facilities USING
GIST(location);
CREATE INDEX idx_inventory_lookup ON inventory(facility_id,
blood_group, resource_type, expiry_timestamp);
CREATE INDEX idx_transfer_requests_status ON
transfer_requests(status);
```

5. Machine Learning Pipeline Specification



5.1 Dataset Preprocessing (NHAMCS_Preprocessing_and_Training.ipynb)

1. **Ingestion:** Load emergency department microdata files from the CDC NHAMCS repository.
2. **Target Variable Construction:** Define daily aggregated consumption $Y_{d,g}$ where d is calendar date and g is blood product group, counting transfusions and urgent surgical interventions.
3. **Feature Transformations:**
   - Cyclical temporal encoding: $\sin\left(\frac{2\pi \cdot day}{7}\right)$, $\cos\left(\frac{2\pi \cdot day}{7}\right)$, $\sin\left(\frac{2\pi \cdot month}{12}\right)$, $\cos\left(\frac{2\pi \cdot month}{12}\right)$.
   - Historical rolling windows: 1-day lag, 7-day rolling mean, 14-day rolling standard deviation.

5.2 Model Architectures & Evaluation Metrics

- **Prophet Parameters:**
  - growth = 'linear', yearly_seasonality = True, weekly_seasonality = True, daily_seasonality = False.
  - interval_width = 0.80 (80% confidence interval band).
  - Metric: Mean Absolute Scaled Error ($MASE < 1.15$).
- **XGBoost Classifier Parameters:**
  - objective = 'binary:logistic', max_depth = 5, learning_rate = 0.05, n_estimators = 250.
  - Class balance: scale_pos_weight tuned to counteract sparse stockout incidents.
  - Metric: Area Under the Precision-Recall Curve ($PR - AUC \geq 0.82$) and Recall ≥ 0.90 on the minority stockout class.

6. FastAPI Backend & API Contracts

6.1 Endpoints Specification

POST /api/v1/triage/ingest

Receives unstructured text and returns the parsed Pydantic schema.

JSON

```
// Request
{
  "raw_text": "Need 4 units of O-negative blood immediately at
Apollo Greams Rd for an incoming Level 1 trauma victim.",
  "source": "radio"
}

// Response (200 OK)
{
  "status": "success",
  "data": {
    "resource_type": "PRBC",
    "blood_group": "O-",
    "units_requested": 4,
    "urgency_tier": 1,
    "target_facility_name": "Apollo Greams Rd",
    "confidence_score": 0.96
  }
}
```

POST /api/v1/agent/solve-dispatch

Invokes the LangChain ReAct agent to compute viable donor paths.

JSON

```
// Request
{
  "destination_facility_id":
"c71a399f-07da-4e4b-91c2-559d81373516",
  "resource_type": "PRBC",
  "blood_group": "O-",
  "units_requested": 4,
  "urgency_tier": 1
}

// Response (200 OK)
{
  "transfer_request_id": "84d59a58-6923-41bb-a5a4-9e32f053dc9a",
```

```
  "recommendations": [
    {
      "rank": 1,
      "origin_facility_id": "993a2164-3e9a-4131-b840-798889da3b2b",
      "origin_facility_name": "Regional Blood Transfusion Center",
      "priority_score": 4.821,
      "distance_km": 6.4,
      "stockout_risk": 0.04,
      "estimated_transit_minutes": 18,
      "reasoning_trace": "Candidate holds 28 units of O- PRBC with
zero projected 72h stockout risk. Transit time (18 mins) falls well
within viability envelope. Quality score is 98.4/100."
    }
  ],
  "hitl_status": "pending_approval"
}
```

PATCH /api/v1/dispatch/{transfer_id}/action

Executes human verification on the recommended transfer.

JSON

```
// Request
{
  "action": "approve", // or "reject"
  "approved_by_user_id": "e6a2dc90-1c5e-49b8-b4b9-389f41b9d401",
  "rejection_reason": null
}

// Response (200 OK)
{
  "transfer_id": "84d59a58-6923-41bb-a5a4-9e32f053dc9a",
  "status": "in_transit",
  "courier_dispatch_broadcast": true
}
```

7. Python Implementation Codebase Skeletons

7.1 LangChain Agent & Tool Wrappers (agent_service.py)

Python

```
import os
import json
from typing import Dict, Any, List
from pydantic import BaseModel, Field
from langchain.tools import tool
from langchain_openai import ChatOpenAI
from langchain.agents import create_react_agent, AgentExecutor
from langchain_core.prompts import PromptTemplate
import psycopg2
from psycopg2.extras import RealDictCursor

# 1. Parameter Schemas
class InventoryLookupArgs(BaseModel):
    blood_group: str = Field(description="Blood group needed, e.g.,
'O-', 'A+'")
    resource_type: str = Field(description="Resource type, e.g.,
'PRBC', 'Platelets'")
    target_facility_id: str = Field(description="UUID of requesting
facility")
    radius_km: float = Field(default=30.0, description="Search
radius in kilometers")

# 2. Tool 1: PostGIS Proximity & Inventory Lookup
@tool("lookup_candidate_inventory",
args_schema=InventoryLookupArgs)
def lookup_candidate_inventory(blood_group: str, resource_type:
str, target_facility_id: str, radius_km: float = 30.0) -> str:
    """Queries Postgres/PostGIS for donor facilities within radius
holding unexpired units."""
    conn = psycopg2.connect(os.environ["DATABASE_URL"])
    cursor = conn.cursor(cursor_factory=RealDictCursor)

    query = """
    WITH target AS (
        SELECT location FROM facilities WHERE id = %s::uuid
    )
    SELECT
        f.id AS facility_id,
        f.name AS facility_name,
        f.quality_score,
        ROUND((ST_Distance(f.location::geography,
```

```
t.location::geography) / 1000)::numeric, 2) AS distance_km,
        COALESCE(SUM(i.units_available), 0) AS
total_available_units
    FROM facilities f
    CROSS JOIN target t
    LEFT JOIN inventory i ON f.id = i.facility_id
    WHERE f.id != %s::uuid
      AND i.blood_group = %s
      AND i.resource_type = %s
      AND i.expiry_timestamp > NOW() + INTERVAL '24 hours'
      AND ST_DWithin(f.location::geography, t.location::geography,
%s * 1000)
    GROUP BY f.id, f.name, f.quality_score, f.location, t.location
    HAVING COALESCE(SUM(i.units_available), 0) > 0
    ORDER BY distance_km ASC;
    """
    cursor.execute(query, (target_facility_id, target_facility_id,
blood_group, resource_type, radius_km))
    results = cursor.fetchall()
    cursor.close()
    conn.close()
    return json.dumps([dict(r) for r in results], default=str)

# 3. Tool 2: XGBoost Stockout Guardrail
@tool("evaluate_stockout_risk")
def evaluate_stockout_risk(donor_facility_id: str, units_to_draw:
int) -> float:
    """Evaluates the probability (0.0 to 1.0) that donating units
causes a source stockout."""
    # In production: Pull features from cache/DB and execute
xgb_model.predict_proba()
    # Mocking inference output for demonstration:
    simulated_risk = 0.08 if units_to_draw <= 5 else 0.45
    return simulated_risk

# 4. Agent Initialization
def initialize_vitalis_agent():
    tools = [lookup_candidate_inventory, evaluate_stockout_risk]
    llm = ChatOpenAI(model="gpt-4o", temperature=0)

    template = """Answer the medical dispatch request using
available tools.
You have access to the following tools:
{tools}
```

```
Use the following format:
Question: the dispatch task
Thought: logical evaluation step
Action: the tool to use, must be one of [{tool_names}]
Action Input: the tool parameters
Observation: the result of the tool
... (this Thought/Action/Action Input/Observation can repeat N
times)
Thought: I have determined the optimal source facility.
Final Answer: A valid JSON string containing recommended_origin_id,
reasoning_trace, and priority_score.

Question: {input}
Thought:{agent_scratchpad}"""

    prompt = PromptTemplate.from_template(template)
    agent = create_react_agent(llm, tools, prompt)
    return AgentExecutor(agent=agent, tools=tools, verbose=True,
handle_parsing_errors=True)
```

8. Non-Functional Requirements & Safety Controls

Category	Criterion	Implementation Specification
Performance	Triage Latency	The Jev/NER extraction layer must return structured JSON within $\leq 500ms$.
Performance	Agent Convergence	The LangChain ReAct optimization loop must converge to a dispatch solution in $\leq 3.5seconds$.
Safety	Depletion Guardrail	No recommendation may pull units from a facility whose 72-hour stockout probability exceeds 0.40.
Safety	Cold-Chain Integrity	IoT sensors must sample temperatures every 60 seconds; any reading $>6^\circ\text{C}$ for RBC trips an alarm.
Security	Privacy & Access	Database adheres to Row Level Security (RLS); Patient names are excluded at the edge, logging only ESI tiers.
Governance	HITL Verification	Autonomous database execution of transit status is blocked; requires admin signature.